

Operating Systems

2520030273

K.Navadeep

SEC-5

WEEK-4

4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

Code

```
kolasani_navadeep@Navadee × + v
GNU nano 8.7.1 child.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t p1, p2;

    p1 = fork();

    if (p1 == 0)
    {
        printf("Child 1 PID: %d\n", getpid());
        sleep(2);
        printf("Child 1 completed\n");
        return 0;
    }

    p2 = fork();

    if (p2 == 0)
    {
        printf("Child 2 PID: %d\n", getpid());
        sleep(4);
        printf("Child 2 completed\n");
        return 0;
    }

    printf("Parent PID: %d\n", getpid());

    wait(NULL);
    printf("One child completed using wait()\n");

    waitpid(p2, NULL, 0);
    printf("Child 2 completed using waitpid()\n");

    return 0;
}
```

output

```
kolasani_navadeep@Navadeep-Vitus:~$ nano child.c
kolasani_navadeep@Navadeep-Vitus:~$ gcc child.c
kolasani_navadeep@Navadeep-Vitus:~$ ./a.out
Parent PID: 813
Child 2 PID: 815
Child 1 PID: 814
Child 1 completed
One child completed using wait()
Child 2 completed
Child 2 completed using waitpid()
kolasani_navadeep@Navadeep-Vitus:~$ |
```